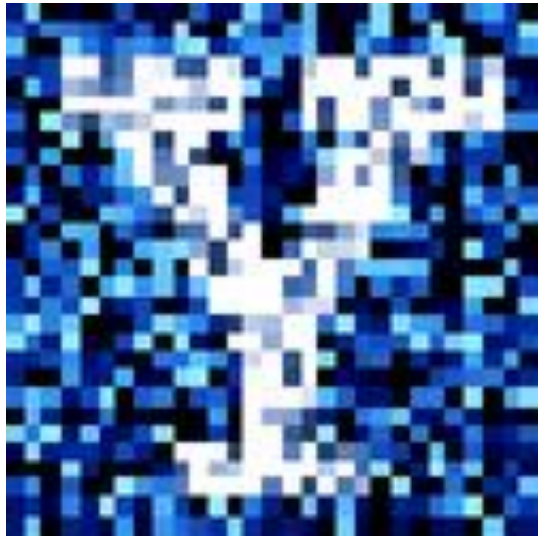
YData: Introduction to Data Science



Lecture 22: Classification

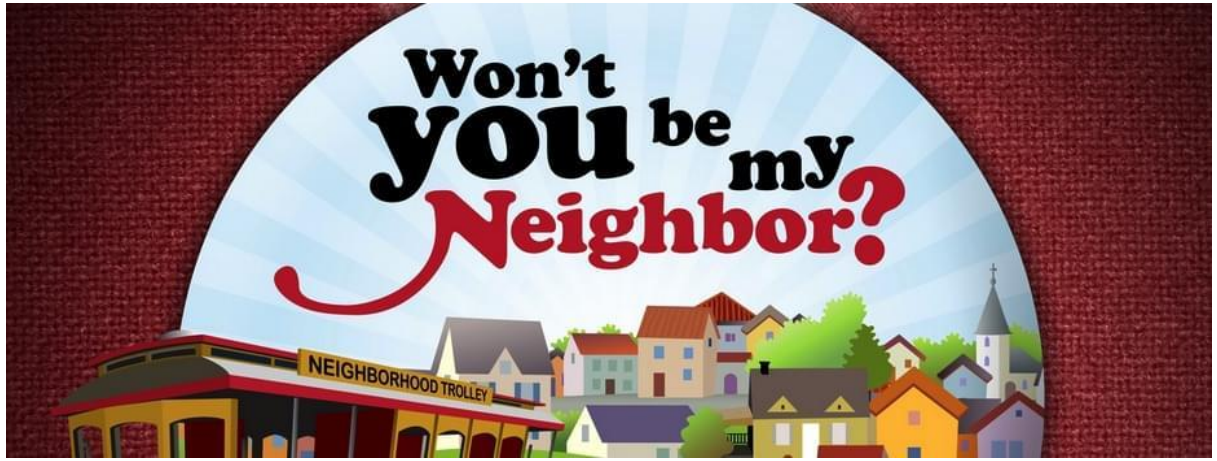
Overview

KNN classifier

Cross-validation

Other classifiers

Building our own KNN classifier



Project timeline

Friday, April 10th

- Projects are due on Gradescope at 11pm
- Email a **pdf** of your project to your peer reviewers
  - A list of whose paper you will review is posted on Canvas
  - Fill out the draft reflection on Canvas

Friday, April 17th

- A template for doing your review has been posted
- Jupyter notebook files with your reviews need to be emailed to the authors
- A pdf containing all three reviews needs to be uploaded Gradescope

Friday, April 24th

- Project is due on Gradescope
- Add the peer reviews of your project to the Appendix of your project

Homework 9 has been posted

- It is due Sunday April 19th

Peer review of projects

A Jupyter notebook template for reviewing projects can be downloaded using:

```
import YData
YData.download_class_file('reviewer_template.ipynb', 'homework')
```

A rubric showing how points should be deducted is on Canvas

- Use this rubric when scoring projects

Please email Shivam if you run into any difficulties

Let’s quickly look at the reviewer template in Jupyter

Review of Classification

Prediction: regression and clasification

We “learn” a function f

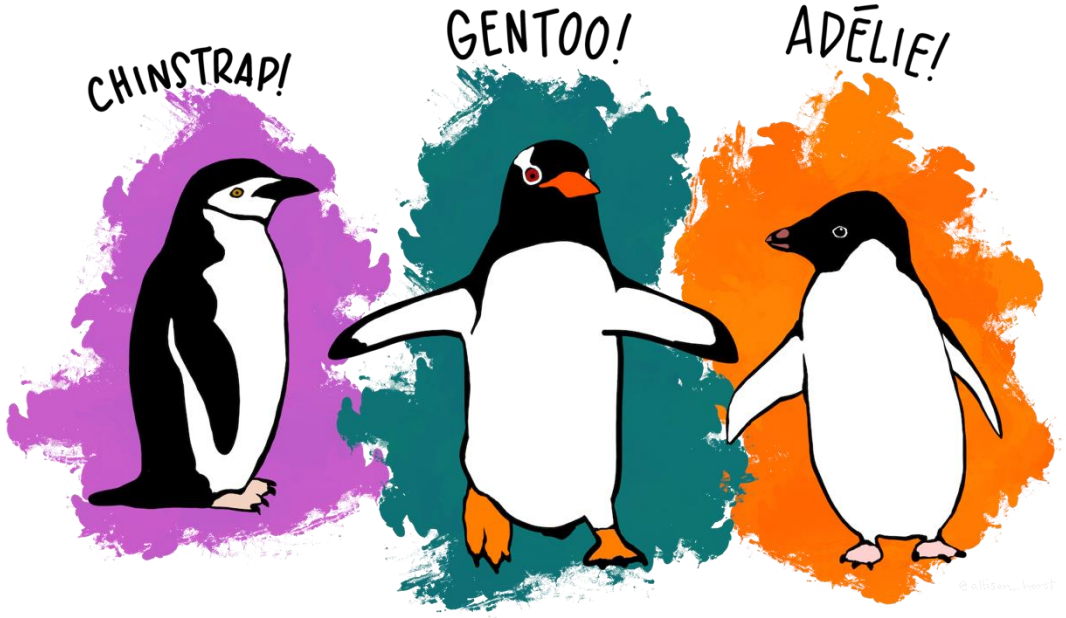
- f(**x**) ⟶ y

Input: **x** is a data vector of "features"

Output:

- Regression: output is a real number (y ϵ R)
- Classification: output is a categorical variable y_k

Example: Penguin species



What are the features and labels in this task?

- Labels (y): Chinstrap, Gentoo, Adelie
- Features (X): Flipper length, bill length, body mass, …

Classifiers

Classifiers take a list/array of features X, and return a predicted label y

There are many different types of classifiers

- Decision Trees, Support Vector Machines, Logistic regression, Neural Networks, etc.

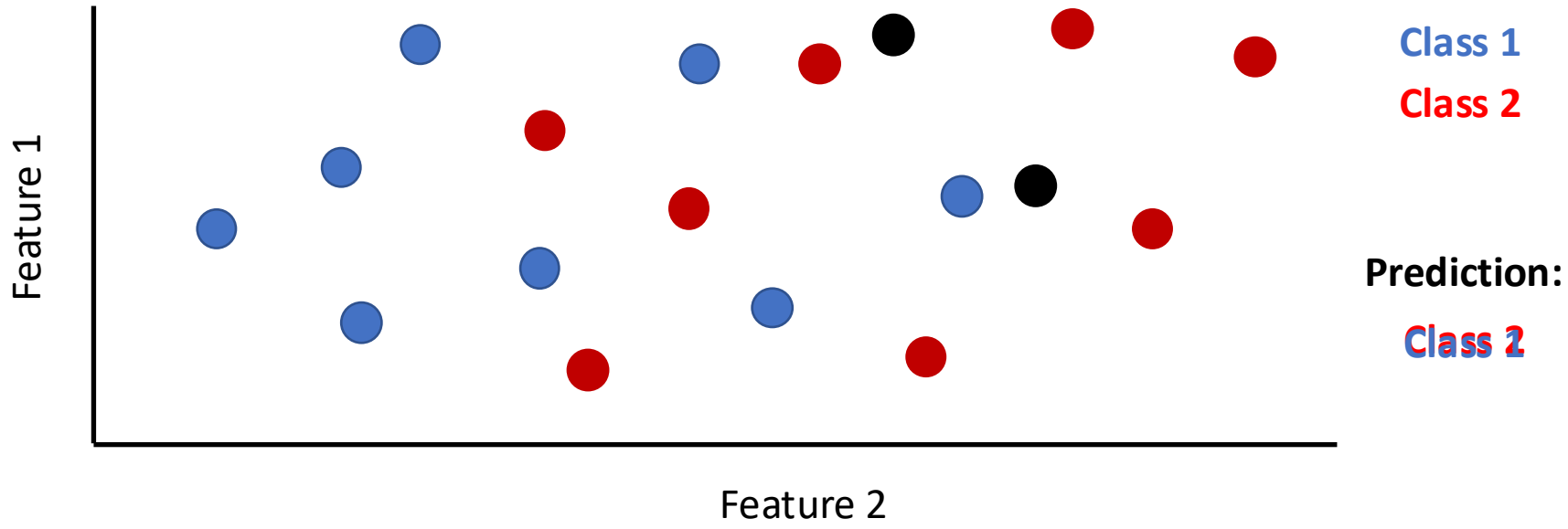
The classification process always involves two steps:

1. **Training** the classifier to learn the relationship between features (X) and labels (y) (from many examples)
   - This is done using the .fit() method in scikit-learn
2. **Using** the classifier to predict a label y, from features X for a new data point
   - This is done using the .predict() method in scikit-learn

K-Nearest Neighbor Classifier (KNN)

Training the classifier: Store all the features with their labels

Making predictions: The label of closest k training points is returned



Distance between two points

Suppose we have two data points p_0 and p_1 and we want to compute the distance between these points

If the data points have two features x and y

- Our data is: $p_0 = [x_0, y_0]$ and $p_1 = [x_1, y_1]$
- The distance between the two data points is:

$$D = \sqrt{(x_0 - x_1)^2 + (y_0 - y_1)^2}$$

If the data points have three features x, y and z

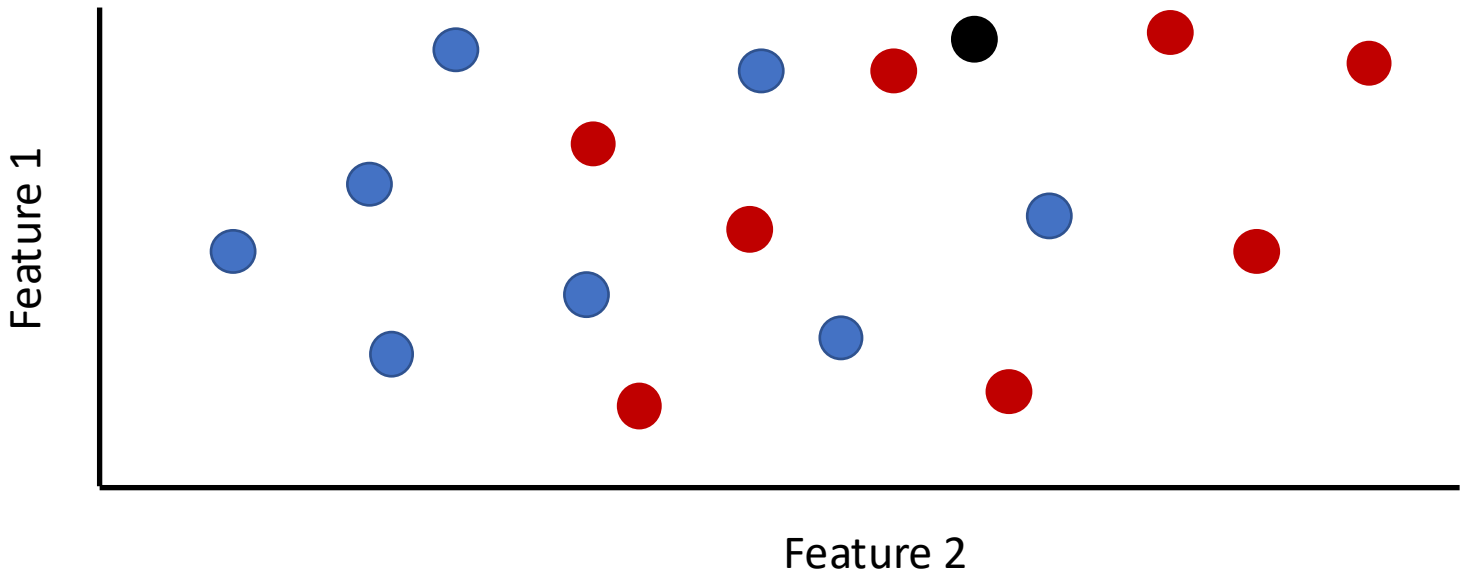
- Our data is: $p_0 = [x_0, y_0, z_0]$ and $p_1 = [x_1, y_1, z_1]$
- The distance between the two data points is:

$$D = \sqrt{(x_0 - x_1)^2 + (y_0 - y_1)^2 + (z_0 - z_1)^2}$$

Finding the k Nearest Neighbors (k ≥ 1)

To classify a point:

- Find its k nearest neighbors
- Take a majority vote of the k nearest neighbors to see which of the two classes appears more often
- Assign the point the class that wins the majority vote



KNN classifiers using scikit-learn

We can fit and evaluate the performance of a KNN classifier using:

```
knn = KNeighborsClassifier(n_neighbors = 1)        # construct a classifier

knn.fit(X_features, y_labels)       # train the classifier

y_test_preditions = knn.predict(X_test_features)  # make predictions

np.mean(y_test_preditions == y_test_labels)        # get accuracy
```

Let’s explore this in Jupyter!

Evaluation

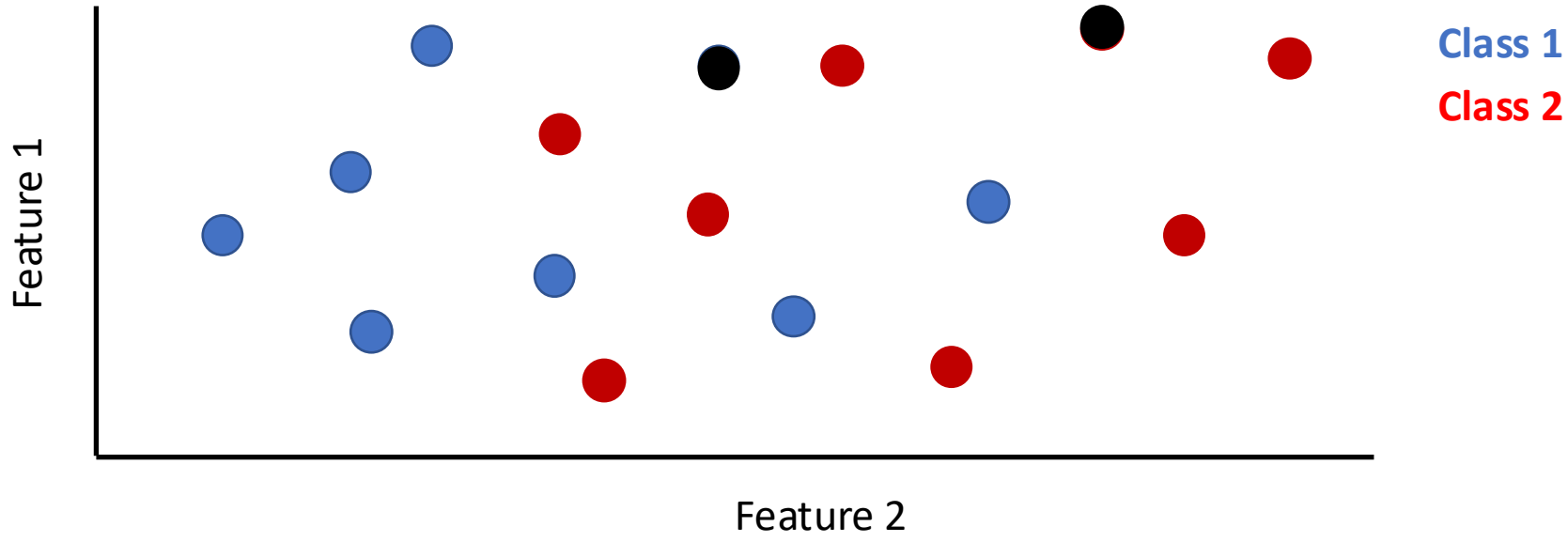
Training and test accuracy

Q: What would happen if we tested the classifier using the same data with k = 1?

A: We would have 100% accuracy

Q: Would this indicate that the classifier is good?

- A: No!



Cross-validation

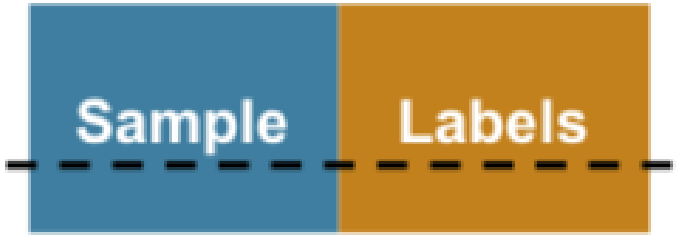
To assess how accurate the predictions of a classifier are, we need to split our data into a training set and test set

We fit the classifier on the training set

- i.e., we learn the relationship between labels (y) and features (x) on the training set

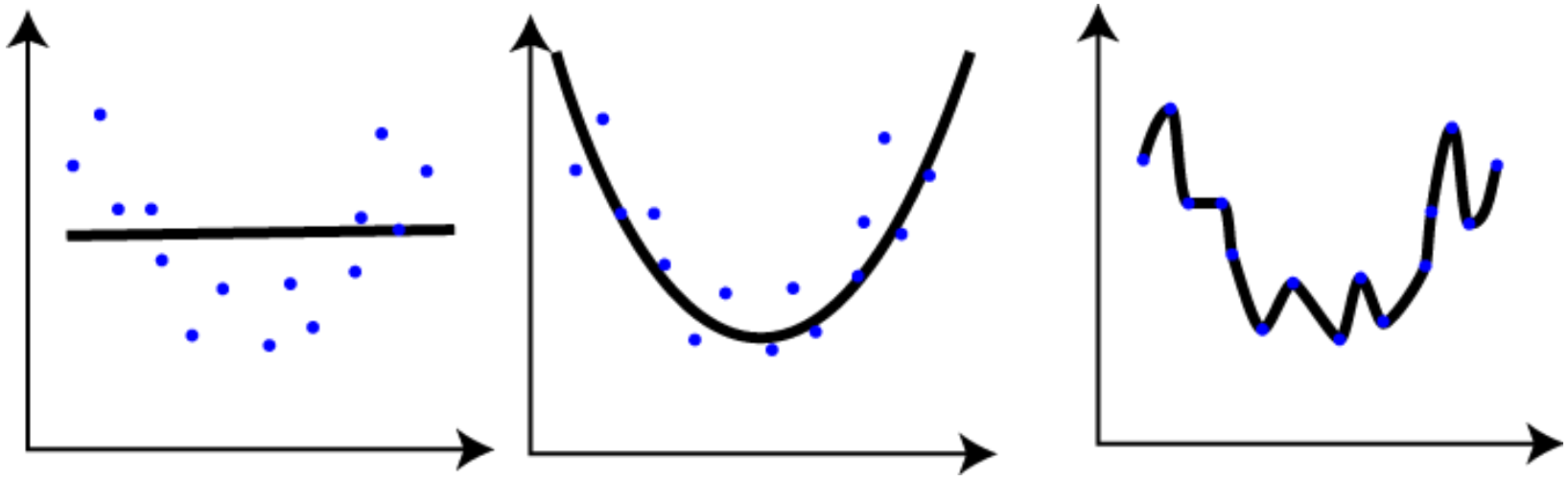
We assess the classifier's performance on the test set

The process of fitting your classifier on a training set and evaluation your classifier on a test set is called cross-validation



Cross-validation can help prevent overfitting!

Overfitting



Overfitting

If our classifier has over-fit to the training data then:

a. We might not have a realistic estimate of how accurate its predictions will be on new data

b. There might be a better classifier that would not over-fit to the data and thus can make better predictions

What we really want to estimate is how well the classifier will make predictions on new data, which is called the **generalization (or test) error**

Overfitting song…

k-fold cross-validation

Are there any downsides to using half the data for training and half for testing?

Since we are only using half the data for training, potentially can build a better model if we used more of our data

- Say 90% of our data for training and 10% of testing

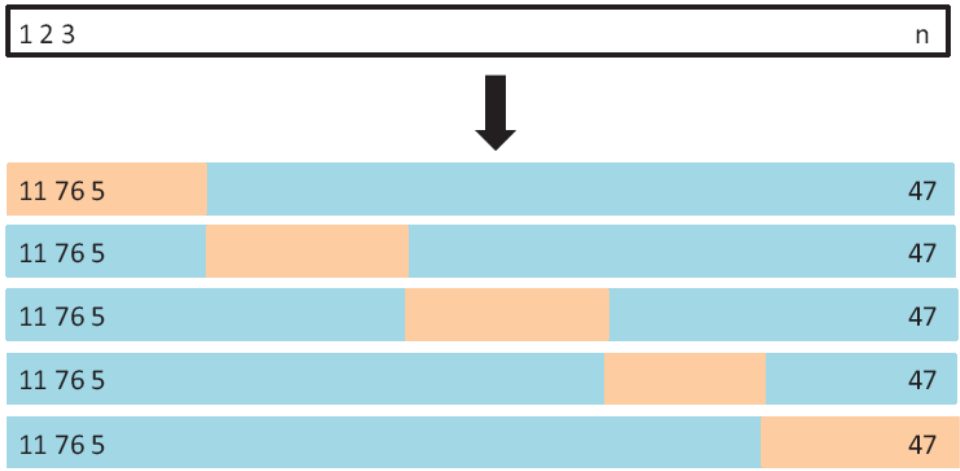
Problem: Then our estimate of the generalization error would be worse

Solutions?

k-fold cross-validation

k-fold cross-validation

- Split the data into k parts
- Train on k-1 of these parts and test on the left out part
- Repeat this process for all k parts
- Average the prediction accuracies to get a final estimate of the generalization error

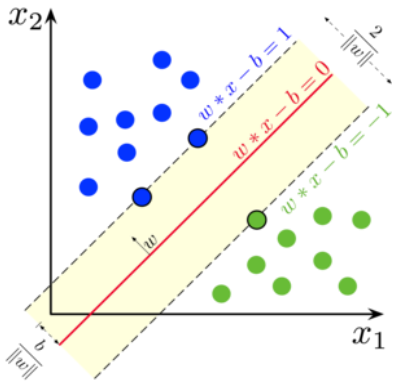


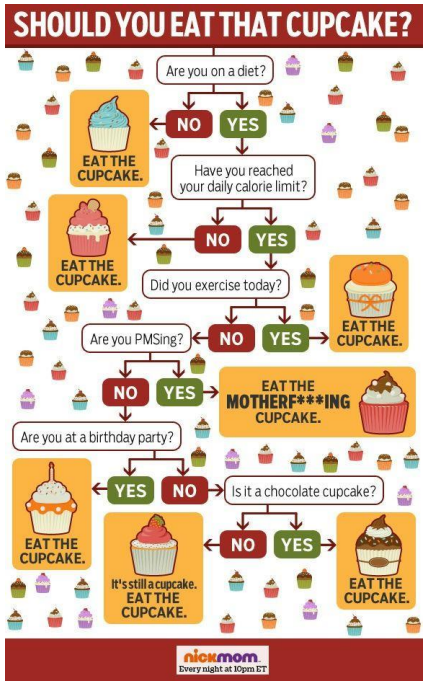
Let’s try this in Jupyter!

Other classifiers

There are many other classification algorithms such as:

- Support Vector Machines (SVM)
- Decision Trees/Random Forests
- Deep Neural Networks
- etc.





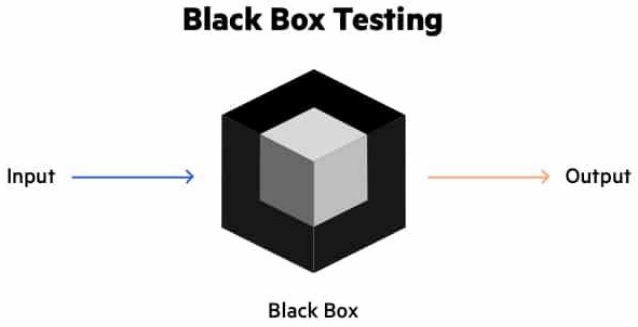
Scikit-learn makes it easy to try out different classifiers get their cross-validation performance

```
svm = LinearSVC()

scores = cross_val_score(svm, X_features,  y_labels, cv = 5)

scores.mean()
```

Let's quickly try this in Jupyter!



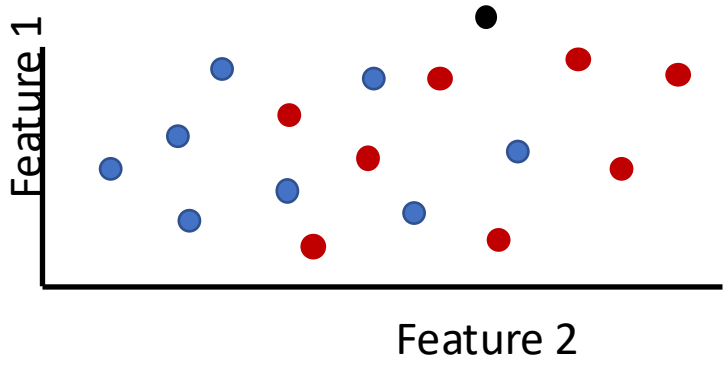
Building a KNN classifier

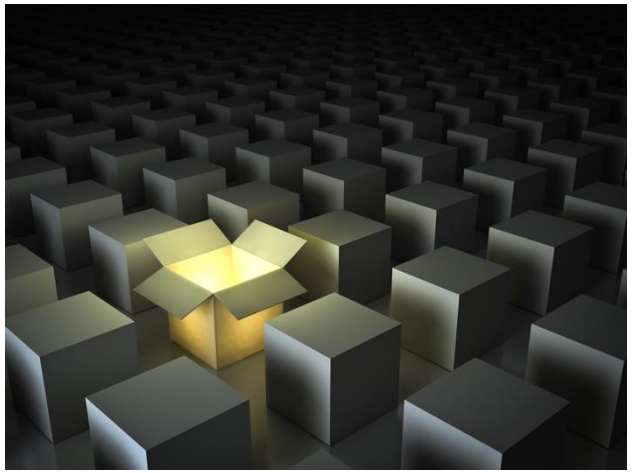
Building the KNN classifier

So far we have used a KNN classifier

- and we have some idea of how it works

Let's now see if we can write to to implement the classifier ourselves…





Steps to build a KNN classifier

We build our KNN classifier by creating a series of functions…

$$D = \sqrt{(x_0 - x_1)^2 + (y_0 - y_1)^2 + (z_0 - z_1)^2}$$

1. euclid_dist(x1, x2)
    - Calculates the Euclidean distance between two points x1 and x2

2. get_labels_and_distances(test_point, X_train_features, y_train_labels)
    - Finds the distance between a test point and all the training points

3. classify_point(test_point, k, X_train_features, y_train_labels)
    - Classifies one test point by returning the majority label of the k closest points

4. classify_all_test_data(X_test_data, k, X_train_features, y_train_labels)
    - Classifiers all test points

Let's continue exploring this in Jupyter!